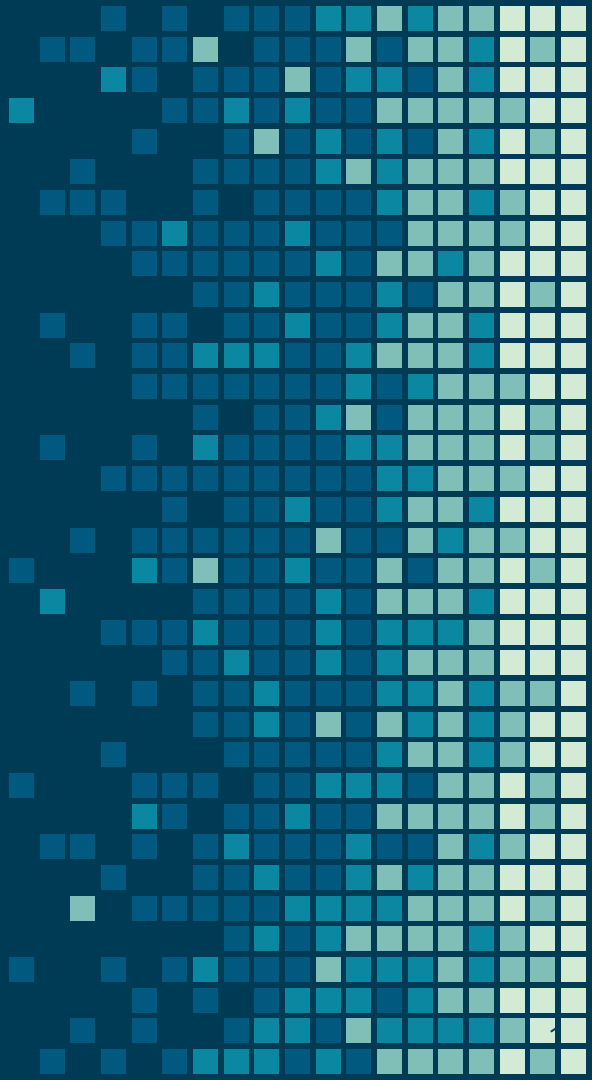
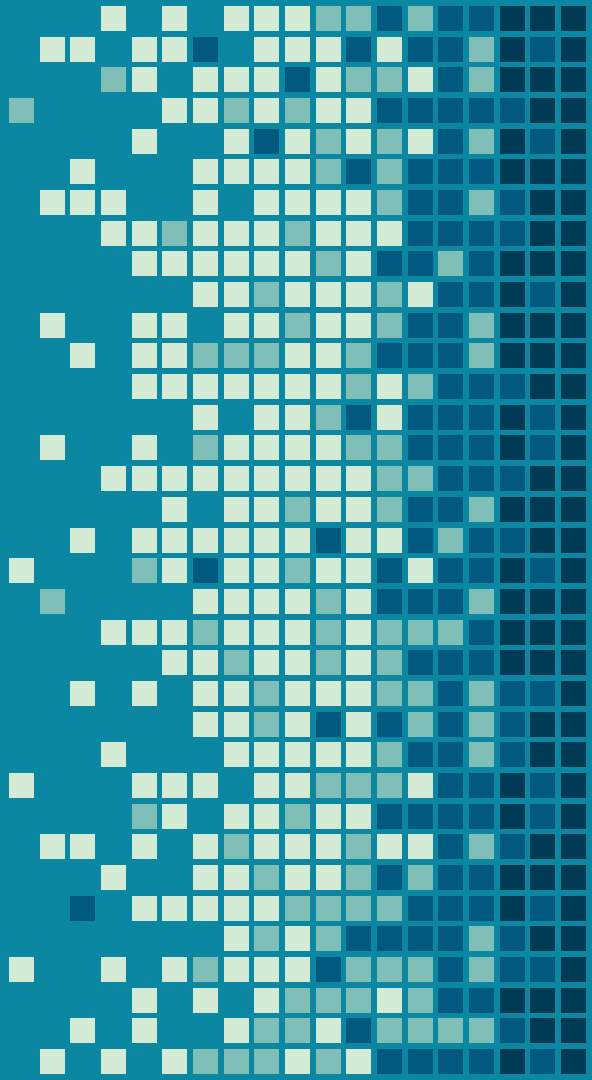
EMBEDDED CONTROL

Prof. Luca De Cicco
luca.decicco@poliba.it
https://c3lab.poliba.it/LDC



I/O synchronization and Interrupts handling

> *One of the most frequently asked questions in Embedded Systems design job interviews is related to* ***interrupts*** *and how to handle them*



Where are we?

- We have defined what we need to implement an embedded control loop
- We have studied ARM Cortex-M core architecture and programming model basics
- We have introduced STM32 ARM Cortex-M based microcontrollers
- We have seen how to access programmatically a GPIO both at register level and using HAL
- **We now need to learn how to access I/O peripherals both internal and external of the MCU**
- Then we will focus on learning the specifics of internal I/O peripherals of STM32 MCUs using the HAL

I/O synchronization mechanisms



Accessing Input/Output devices

- Embedded control loops need to **access input** (read from sensors) and **output devices** (actuate control) from the MCU
- The model used to access I/O devices inherently influences the firmware design strategy
- Knowing the implications and the I/O access strategy is a fundamental knowledge to develop firmwares that **meet requirements** and are **safe** to operate
- Typical trade-off:
  - **simple** to implement strategy means **less performing**
  - conversely **more complex** strategies **perform better** (but might entail **safety** issues)

I/O basic performance metrics: latency

- **Latency**: time between when I/O device needs service and when the service is actually initiated
- Latency = Hardware Latency + Software Latency
  - HW latency: due to digital gates delay
  - SW latency: due to software components (sometimes difficult to predict)
- **Real-time system** = System with bounded I/O latency! (we have a measure of the worst case latency)

I/O basic performance metrics: bandwidth

- **Bandwidth** = data/second (bytes/s) the I/O device can read/write per unit of time
- Bandwidth can be:
  - HW limited: HW bandwidth is typically known and specified in datasheets (f.i., max theoretical I2C speed ~3.2 Mbps)
  - Software limited: due to issues in handling data from the HW device
- If bandwidth is limited by software we are typically doing something wrong when accessing the HW

How long I/O does it take?

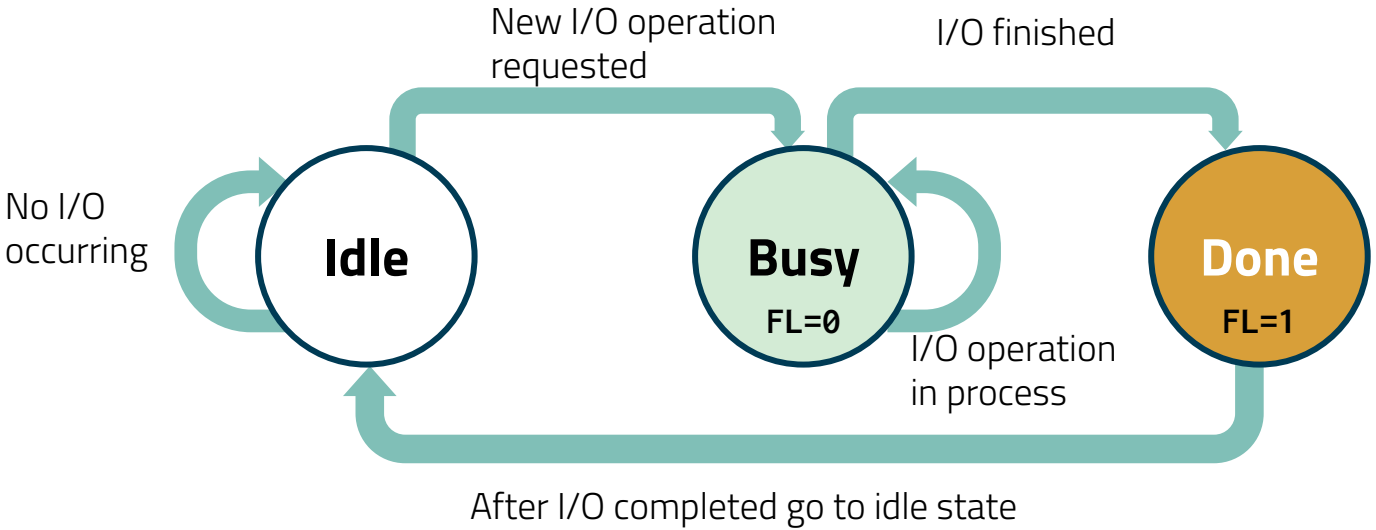
- Depends on latency, bandwidth, and data size
- Rough (optimistic) estimate:

```
T = Latency + data_size/Bandwidth
```

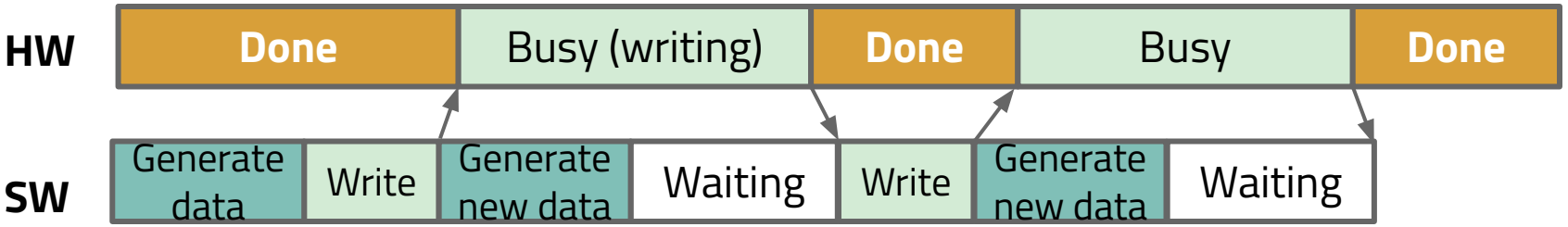
- Latency is roughly fixed, the second component is variable due to changing data size
- It's possible to estimate I/O operation length
- However, it's not wise to use such information to schedule I/O operations

Synchronizing SW with state of HW I/O

- Hardware I/O device can be assumed to be in one of three states
- Software needs to know when these transition occur (you cannot "superpose" different operations, recall Tartarus Challenge)
- Typically (not always) **I/O is slower than software** (CPU)

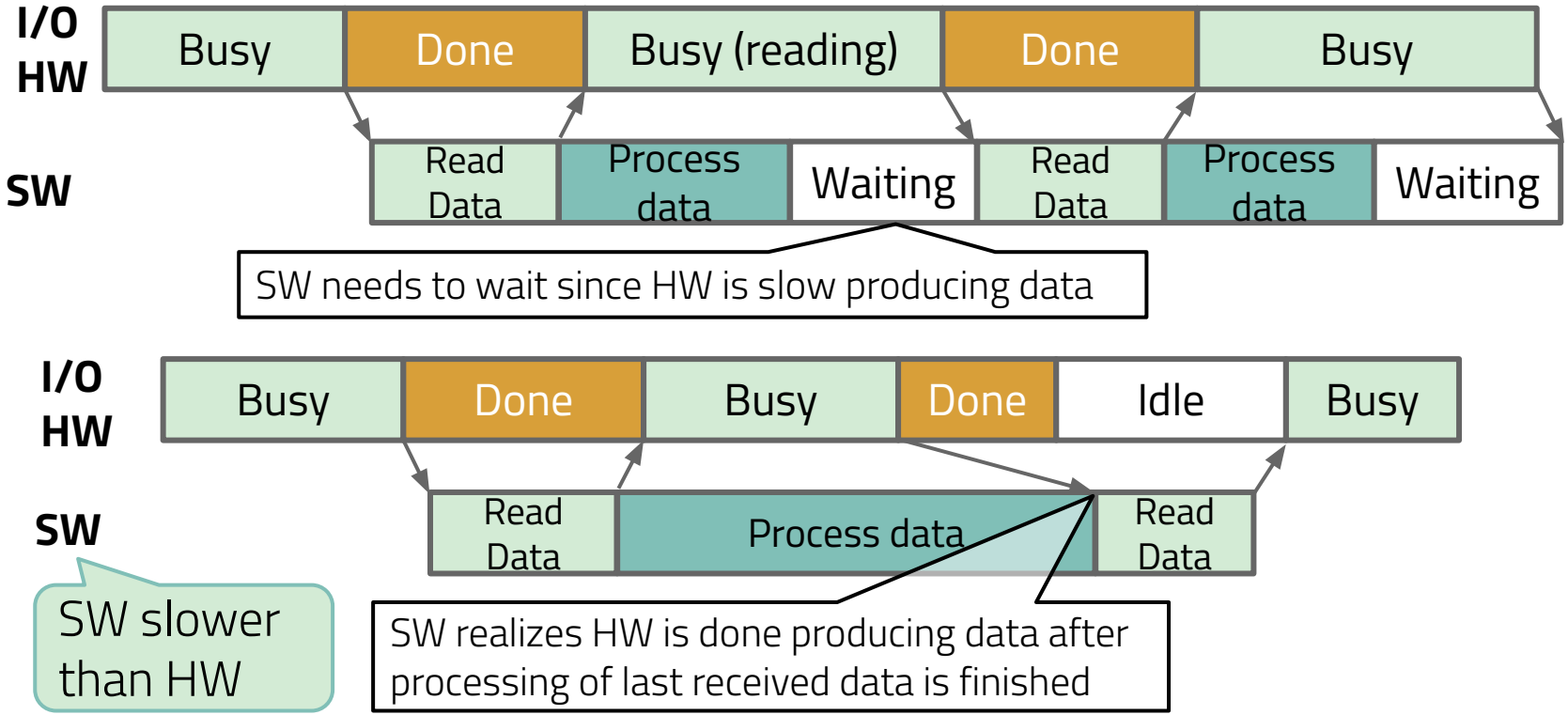


Output synchronization (writing to actuator)



- In this example software **generates** data then asks HW to **write** data and then SW goes idle **waiting** HW to complete the output operation
- Notice: HW never goes in idle state
- This is a typical situation where SW computation is faster than HW input/output

Input synchronization (reading from sensor)



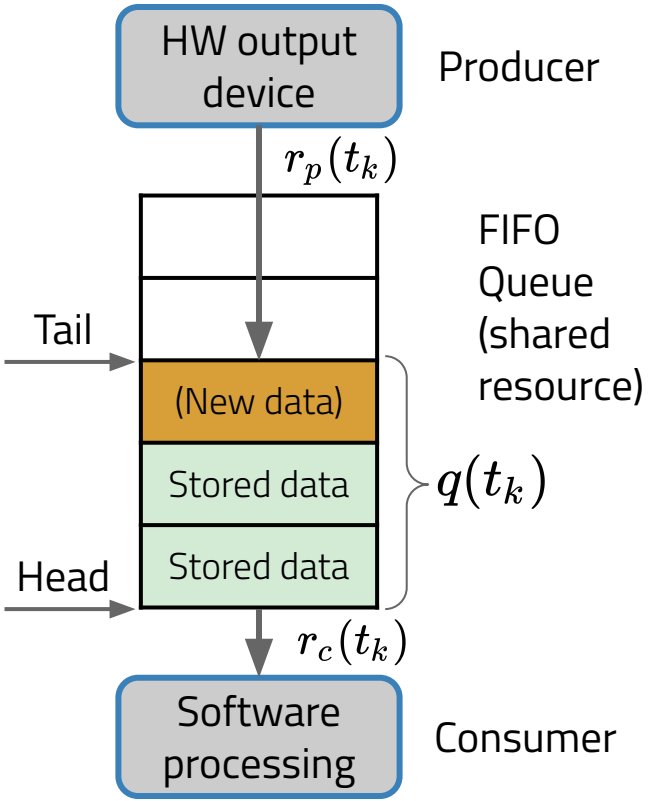
I/O bound vs CPU bound applications

- When software needs to go idle waiting for HW to become ready for I/O we define the application **I/O bound**
- Conversely, when hardware needs to go idle waiting for SW to become ready to perform a new I/O operation we define the application **CPU bound**
- Typically, HW I/O is slower than CPU processing
- **WARNING:** When the application is CPU bound, it is a sign you either need a faster processing (CPU) or you need to rethink your I/O management pipeline

Unbuffered vs Buffered I/O

- The examples given are all “unbuffered” in the sense that I/O data is not temporarily stored in RAM before being processed by software
- Unbuffered configuration leads to HW or SW to wait each other
- Buffers can be used to temporarily absorb HW or SW bandwidth fluctuations
- Buffered Output operation example: HW stores data in a buffer independently (HW always busy), SW consumes data from the buffer when buffer is not empty

Buffered I/O - producer-consumer pattern



- A buffer can be modelled as an integrator
- The queue stores the rate mismatch between produced data rate ($r_p(t_k)$ and consumed data rate $r_c(t_k)$)
- $q(t_k) = q(t_{k-1}) + (r_p(t_k) - r_c(t_k))(t_k - t_{k-1})$
- Ideally the queue length should always be 0 (if system well designed)
- $r_c(t_k)$ could slow down due to data processing being slower than normal
- The queue temporarily "absorbs" this rate fluctuation
- **Q**: What happens if SW slower than HW?

Synchronizing HW and SW in I/O operations

Blind cycle:

- software implies a fixed amount of time and assumes I/O will complete after that delay
- No check is done on I/O device status
- Pros: trivial to implement
- Cons: can waste too much computing resources or could read/write data when I/O not idle

Synchronizing HW and SW in I/O operations

Busy-wait (polling mode):

- Software loops until when I/O HW status changes to "Done"
- When loop ends SW reads or writes data from HW
- Pros: easy to implement
- Cons: wastes processing resources exactly for the amount of time required for I/O (better than blind!)
- Used often when the system is quite simple and wasting resources is not an issue
- Recall Tartarus challenge?

Synchronizing HW and SW in I/O operations

Interrupts: (a lot more on interrupts later!)

- When I/O gets ready the software gets immediately notified through an “interrupt”
- Output device example: write data and get notified asynchronously when write operation is finished
- Reading/writing using interrupts does not need to actively checking status of HW, instead we get notified
- Pros: consumes very few CPU cycles (can perform other work in parallel)
- Cons: complicates firmware and adds the “concurrency” issue

Synchronizing HW and SW in I/O operations

Direct Memory Access: (more later in this course)

- Data transfer is done independently using the DMA controller
- The I/O device read/writes directly from/to a memory region
- Software then reads from the known memory region
- Used for high speed data transfers requiring large chunks of data to be transferred (f.i., image sensor)
- Pros: almost zero processing required for I/O
- Cons: similarly to interrupts complicates firmware and adds concurrency issues

Exceptions (interrupts) basics



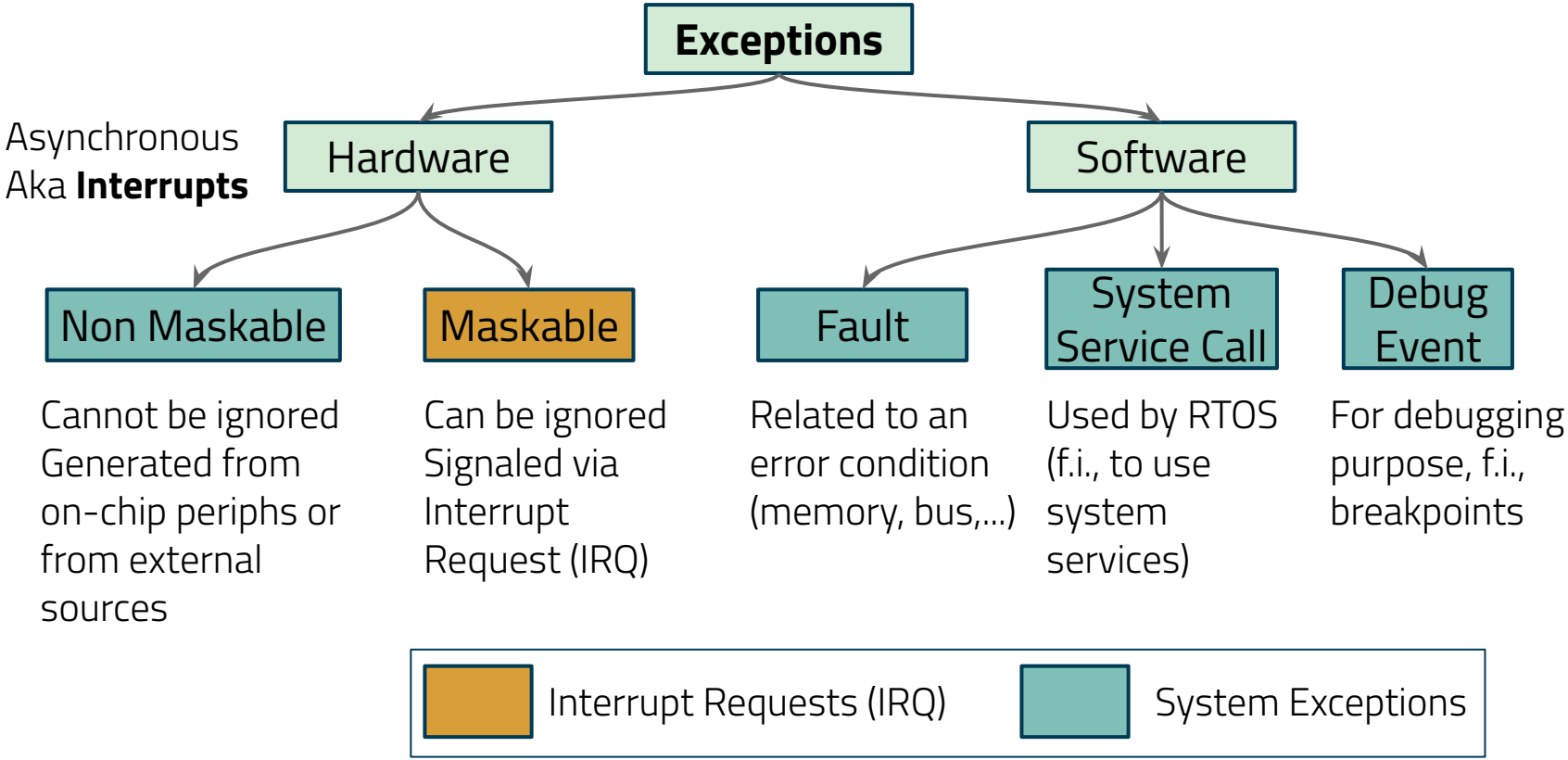
Exceptions*

An **Exception** is an event that causes changes to program flow

- When an exception occurs an **Exception Handler** (a function in the firmware) is called “immediately” to **handle the interrupt** interrupting the normal execution of the main firmware
- A specific hardware in the MCU core handles exceptions
- Several types of exceptions can occur

(*) We are using the ARM terminology, i.e., an interrupt is a special case of an exception

Exceptions classification



Exceptions basics (1)

Example of exceptions:

- Software: division by 0, arithmetic overflow, illegal opcode
- Hardware: I/O operation completion, end of conversion, timer time-out

Use of exceptions:

- **I/O synchronization** (first part of the lecture!)
- Respond to sporadic but critical events (HW faults…)
- **Periodic tasks**: timers generate interrupts that can be used to perform a periodic task (f.i., control law computation, periodic sensors readings)

Interrupt maskability

- **Maskable**
  - these interrupts **can be ignored** by the processor
  - maskable interrupts need to be enabled to interrupt the CPU when occurring
  - Enabling an interrupt = setting an enable bit in a register
- **Non-Maskable Interrupts (NMI)**: these interrupts cannot be ignored
  - NMI requires a special attention (a severe issue)

Exception Handlers and Exception Priority

Exception Handlers or **Interrupt Service Routine (ISR)**

- The function called to "serve" the exception
- Implemented in the firmware
- The implementation of a specific ISR is application-specific and depends on the reason the interrupt was generated

Exceptions can be **prioritized**

- This allows having **multiple pending exceptions** (an exception occurring inside an exception handler)
- When having multiple pending exceptions, higher priority exception is served first

Interrupt Vector and Vector Table

- **Interrupt Vector**: the start address of the exception handler function. Interrupt vector can be seen as a function pointer
- **Vector Table**: table of interrupt vectors that associates an interrupt handler with an interrupt request. It is basically an array of interrupt vectors stored in memory
- Each exception is associated to a number (exception number)
- When the exception occurs the exception number is used to identify the interrupt vector in the vector table
- The processor will then know where to jump to handle the exception

Exception handling flow (HW interrupt ex.)

1. Hardware triggers an interrupt
2. An MCU hardware component detects the interrupt and its type (the exception number)
3. The MCU state is automatically saved
4. The corresponding Interrupt Service Routine (ISR) is called (Vector Table lookup)
5. The Interrupt is acknowledged in the firmware so it is not triggered again (a flag is cleared in a register)
6. The MCU state is restored
7. The execution of the main program is resumed